As you can see, programming languages can have separate local variables with the same name (`spam`) because they are kept in separate frame objects. When a local variable is used in the source code, the variable with that name in the topmost frame object is used.

Every running program has a call stack, and multithreaded programs have one call stack for each thread. But when you look at the source code for a program, you can't see the call stack in the code. The call stack isn't stored in a variable as other data structures are; it's automatically handled in the background.

The fact that the call stack doesn't exist in source code is the main reason recursion is so confusing to beginners: recursion relies on something the programmer can't even see! Revealing how stack data structures and the call stack work removes much of mystery behind recursion. Functions and stacks are both simple concepts, and we can use them together to understand how recursion works.

What Are Recursive Functions and Stack Overflows?

A *recursive function* is a function that calls itself. This *shortest.py* program is the shortest possible example of a recursive function:

Python

```
def shortest():
    shortest()

shortest()
```

The preceding program is equivalent to this *shortest.html* program:

JavaScript

```
<script type="text/javascript">
function shortest() {
    shortest();
}

shortest();
</script>
```

The `shortest()` function does nothing but call the `shortest()` function. When this happens, it calls the `shortest()` function again, and that will call `shortest()`, and so on seemingly forever. It is similar to the mythological idea that the crust of the Earth rests on the back of a giant space turtle, which rests on the back of another turtle. Beneath that turtle: another turtle. And so on, forever.

But this "turtles all the way down" theory doesn't do a good job of explaining cosmology, nor recursive functions. Since the call stack uses the computer's finite memory, this program cannot continue forever, the way an infinite loop does. The only thing this program does is crash and display an error message.

NOTE *To view the JavaScript error, you must open the* browser developer tools. *On most browsers, this is done by pressing* ***F12*** *and then selecting the Console tab.*